# Detailed Technical Design Document - Student 5

**Digital Attestation Architecture (Cosign), Security Testing Plan & Integration Framework**

**Role:** Security Tester & Integration Lead
**Focus:** Digital Attestation & Verification
**Project:** Geo-Bound Vault & DevSecOps Pipeline

## 1. Scope of Responsibilities (Overview & Scope)

Student #5 is responsible for **Software Attestation & Integrity Verification** using Cosign, constructing a **Comprehensive Security Testing Plan (Red Team vs. Blue Team Testing)**, and orchestrating a **Seamless End-to-End System Integration** across all 5 team members' sub-systems.

## 2. Detailed Digital Attestation Design using Cosign

The objective of Cosign (from Sigstore) is to provide an unforgeable digital proof demonstrating that the running application was built and scanned exclusively through the authorized CI/CD pipeline (GitHub Actions) without malicious tampering or backdoor injection.

### A. Cryptographic Keypair Management

- **Keypair Generation:** Execute `cosign generate-key-pair` to generate a private key (`cosign.key`) and a public key (`cosign.pub`).
- **Private Key Security:** Store the private key and passphrase within `GitHub Encrypted Secrets`, exposing it dynamically to the pipeline during execution.
- **Public Key Deployment:** Embed the public key (`cosign.pub`) into the server runtime environment to verify artifact authenticity upon execution.

### B. Pipeline Attestation Workflow (GitHub Actions Integration)

```
# Step 1: Install Cosign CLI in GitHub Actions - name: Install Cosign uses: sigstore/cosign-installer@v3.3.0 # Step 2: Sign Built Artifact / Container Image after Syft & Grype Scans - name: Sign Container Image with Private Key env: COSIGN_PRIVATE_KEY: ${{ secrets.COSIGN_PRIVATE_KEY }} COSIGN_PASSWORD: ${{ secrets.COSIGN_PASSWORD }} run: | cosign sign --key env://COSIGN_PRIVATE_KEY \ --a "builder=GitHubActions" \ --a "repo=${{ github.repository }}" \ --a "sbom=Syft-Verified" \ myapp/geo-vault:latest # Step 3: Attach In-Toto Attestation for Supply Chain Auditing - name: Attach SBOM Attestation run: | cosign attest --key env://COSIGN_PRIVATE_KEY \ --type spdx --predicate sbom.spdx.json myapp/geo-vault:latest
```

### C. Runtime Attestation Verification Logic

Before allowing the encrypted vault to process encryption/decryption requests, the server performs a pre-flight integrity check on the application binary:

```
import subprocess def verify_application_integrity(): try: # Execute cosign verify against the trusted public key result = subprocess.run([ "cosign", "verify", "--key", "/etc/keys/cosign.pub", "myapp/geo-vault:latest" ], capture_output=True, text=True, check=True) return True, "Attestation Validated: Code is authentic and untampered." except subprocess.CalledProcessError: # Trigger immediate system lockdown return False, "SECURITY ALERT: Unverified or Tampered Binary! Access Denied."
```

## 3. Comprehensive Security Testing Plan

Student #5 designed a structured security test matrix covering key operational aspects to validate defense mechanisms and ensure zero unmitigated security vulnerabilities:

| Test Suite | Attack Vector / Test Scenario | Target Component | Expected Defense Result |
|---|---|---|---|
| **Attestation Bypass Test** | Attempting to run a manually altered application binary without a valid Cosign signature. | Runtime Integrity Verification Engine | Execution blocked; system triggers security lockdown and alerts admin. |
| **Pipeline Fail-Safe Test** | Pushing code dependencies containing known High/Critical CVE vulnerabilities. | Grype Scanner in CI/CD Pipeline | Pipeline fails automatically (Fail-Pipeline); binary build aborted. |
| **Geo-Spoofing & Replay** | Simulating mock GPS coordinates or replaying location tokens. | Haversine Formula & Nonce Verification | Access denied; user requested to supply live GPS verification. |
| **Ciphertext Extraction Test** | Attempting direct database reads on SQLite storage files without authentication keys. | AES-256 (GCM Mode) Vault Layer | Data renders as unreadable random ciphertext. |

## 4. End-to-End System Integration Architecture

Serving as the **Integration Lead**, Student #5 coordinates seamless and secure data flows between the modules authored by all team members:

**Component Integration Execution Workflow:**

1. **Student 4** **DevSecOps Pipeline:** Scans codebase with Syft & Grype to generate verified SBOMs.
2. **Student 5** **Digital Attestation:** Signs the built container image with Cosign and generates cryptographic provenance proof.
3. **Student 1** **Request Handling & Crypto Setup:** Receives user request, verifies app attestation, and initializes AES-256 parameters.
4. **Student 3** **Geo-Fence Verification:** Extracts live GPS coordinates, calculates Haversine distance, and approves decryption context.
5. **Student 2** **Vault Storage Access:** Fetches encrypted data from SQLite and delivers decrypted content securely to UI.